

Complexity Analysis Substitution Method



teach me all you know about Complexity Analysis : Substitution

Sure Arham! Let's dive deep into **Complexity Analysis using the Substitution Method** — it's a powerful technique used to **solve recurrence relations**, which often arise in divide-and-conquer algorithms (like Merge Sort, Binary Search, etc.).

◆ What Is the Substitution Method?

The **Substitution Method** (also known as the **Guess and Verify Method**) is used to:

Find the time complexity (Big-O) of a recursive algorithm by:

1. **Guessing** the form of the solution (usually Big-O).
2. **Using mathematical induction** to **prove** that the guess is correct.

◆ General Steps in Substitution Method

Let's say we have a recurrence:

$$T(n) = 2T(n/2) + n$$

✓ Step 1: Guess the solution

We try to guess the time complexity of the recurrence. For example, we might guess:

$$T(n) = O(n \log n)$$

✓ Step 2: Prove it using induction

We want to prove that:

$$T(n) \leq c \cdot n \log n \quad \text{for some constant } c > 0 \text{ and for all } n \geq n_0$$

Start with the recurrence:

r

$$T(n) = 2T(n/2) + n$$

Apply the inductive hypothesis (assume for smaller input sizes):

scss

$$T(n/2) \leq c \cdot (n/2) \log(n/2)$$

So,

r

$$\begin{aligned} T(n) &\leq 2 * [c \cdot (n/2) \log(n/2)] + n \\ &= c \cdot n \log(n/2) + n \end{aligned}$$

Now, use this identity:

arduino

$$\log(n/2) = \log(n) - \log(2) = \log(n) - 1$$

Then,

r

$$\begin{aligned} T(n) &\leq c \cdot n [\log(n) - 1] + n \\ &= c \cdot n \log(n) - c \cdot n + n \\ &= c \cdot n \log(n) - (c \cdot n - n) \end{aligned}$$

For this to be $\leq c \cdot n \log(n)$, we want:

r

$$(c \cdot n - n) \geq 0 \Rightarrow c \geq 1$$

So if $c \geq 1$, then the guess holds:

r

$$T(n) = O(n \log n)$$

◆ When To Use Substitution Method?

Use it when:

- You're given a recurrence like:
 - $T(n) = T(n/2) + 1$
 - $T(n) = 2T(n/2) + n$

- $T(n) = 3T(n/4) + n$
- You need to **prove** bounds like $O(n \log n)$, $O(n)$, etc.

◆ Tips and Tricks

✓ Common Patterns

Recurrence	Guess	Explanation
$T(n) = T(n - 1) + 1$	$O(n)$	Linear recursion
$T(n) = T(n/2) + 1$	$O(\log n)$	Binary search-type
$T(n) = 2T(n/2) + n$	$O(n \log n)$	Merge Sort
$T(n) = 2T(n/2) + 1$	$O(n)$	Divide and conquer with small combine time

◆ Example 2

```
r
T(n) = 3T(n/4) + n
```

Step 1: Guess

Try: $T(n) = O(n)$

Step 2: Verify with substitution

Assume:

```
scss
T(n/4) ≤ c · (n/4)
```

So,

```
r
T(n) = 3T(n/4) + n
      ≤ 3 · c(n/4) + n
      = (3c/4) · n + n
      = n[(3c/4) + 1]
```

We want this $\leq c \cdot n \Rightarrow$

```
r
(3c/4) + 1 ≤ c
⇒ 1 ≤ c - (3c/4)
⇒ 1 ≤ c(1 - 3/4) = c(1/4)
⇒ c ≥ 4
```

So if $c \geq 4$, the guess holds.

✓ Hence, $T(n) = O(n)$

◆ Pros and Cons of Substitution Method

✓ Pros:

- Intuitive once practiced
- Works for proving **both upper and lower bounds**

✗ Cons:

- Requires **guessing** (not always obvious)
- Proof steps need to be carefully done (induction)

◆ Bonus: Template for Using Substitution in Exams

plaintext

Assume $T(n) \leq c \cdot f(n)$ for all $n < N$ (Inductive Hypothesis)

$$\begin{aligned} T(n) &= a \cdot T(n/b) + g(n) \\ &\leq a \cdot [c \cdot f(n/b)] + g(n) \\ &= \text{expression involving } c, f(n), g(n) \end{aligned}$$

Use identities/log rules to simplify

Show $T(n) \leq c \cdot f(n)$ for some constant c

Hence, $T(n) = O(f(n))$ [Proved by induction]

Would you like **practice problems** or a **PDF formula sheet** for substitution and other methods (recursion tree, master theorem)?